

Asymptotic after the first nine reviews

Proof-context leaks, codomain loss,
and native-representation boundaries

Prepared for Vladimir Reshetnikov
9 September 2026

Repository `VladimirReshetnikov/Asymptotic`

Pinned revision `921387e5ba1239bfda96e63e64e89bf63d9c41e6`

Native audit kernel `Wolfram Language 15.0.0, Linux x86-64`

Principal results. Three materially incremental findings were reproduced in a native Wolfram kernel. Ambient assumptions can turn an invalid source-domain check into a positive inverse certificate. A nonreal algebraic coefficient can survive admission, disappear into a remainder, and cause a false trigonometric error bound. A one-slot native series view can violate machine-integer index constraints despite the recent dense-allocation fix.

This article includes mathematical counterexamples, a feature comparison with built-in Wolfram Language, a prior-review cross-reference, bounded reproduction code, and focused candidate changes. Individual repair mechanisms were tested; the complete upstream suite and the delivered combined regression file were not run. No repository files were changed.

Contents

1	Executive assessment	2
1.1	Recommendation	2
1.2	Positioning relative to Wolfram Language	2
2	Snapshot, methodology, and prior-review filtering	3
2.1	What is pinned and what was executed	3
2.2	Evidence classes and limits	3
2.3	How duplication was avoided	4
3	Architecture and comparison with built-in capabilities	4
3.1	The ordinary mathematical representation	4
3.2	Selected extensions, not a universal transseries field	5
3.3	Capability matrix	5
3.4	Executed overlap: polynomial inversion	6
3.5	Executed overlap: a Dirichlet scale	6
4	N01: Ambient assumptions can invalidate a positive certificate	7
4.1	The counterexample	7
4.2	What is correct, and what is false	8
4.3	Root cause: positional assumptions are not isolation	8
4.4	A revealing differential control	9
4.5	Focused repair and validation	9
4.6	Acceptance criteria	9
5	N02: Nonreal admission followed by a false real-only error transport	10
5.1	An exact coefficient without an explicit complex literal	10
5.2	The public calculation	10
5.3	Proof that the remainder is wrong	11
5.4	Source path and the lost information	11
5.5	A focused guard, with its compatibility caveat	12
5.6	Whole-germ codomain evidence	12
6	N03: Native index range is independent of sparse support size	13
6.1	What the earlier repair already accomplished	13
6.2	Two one-slot examples	13
6.3	The precise conversion obligation	14
6.4	Impact and severity	14
6.5	Focused range guard	14
6.6	Performance implication beyond this example	15
7	A development plan derived from the counterexamples	15
7.1	First milestone: make verification context-independent	15
7.2	Second milestone: establish a real-germ boundary	16
7.3	Third milestone: make the native view independently fallible	16
7.4	A compact regression matrix	16
7.5	Performance work should preserve these proof obligations	17
8	What this audit does not add to the backlog	17
8.1	Known issues that are not new findings here	17
8.2	Corrections to overly broad assurances	18
8.3	No unsupported generalization from the findings	18
9	Artifact guide and reproducibility	18
9.1	Files in the archive	18
9.2	Run the native probes on an explicit local package	19
9.3	Generate and review the candidate source changes	19
9.4	Independent checks and article build	20
10	Conclusion	20
A	Source coverage and navigation	20
B	Native evidence index	21

1 Executive assessment

1.1 Recommendation

Treat the next stabilization step as a repair of *proof boundaries*, not an expansion of the feature list. The package is useful precisely because it carries more than a finite formula: it records asymptotic scales, remainders, source equations, branch conditions, and selected verification evidence. A defect that changes the truth of those annotations is more consequential than a missing expansion family. The strongest result here is an explicit certificate claiming that the root 2 satisfies the retained source condition $x > 100$. That claim is false, even though the unrestricted equation $x = 2$ has been solved correctly.

The other two findings concern information crossing representation boundaries. In N02, a syntactic exactness check admits a nonreal `Root`; truncation hides that coefficient, and a real-only inequality is then applied to the entire unknown function. In N03, a mathematically valid sparse series is exported to a native representation whose integer specifications cannot encode it. Both are small-input examples. Neither requires a large allocation, a pathological approximation threshold, or an untested conjecture about asymptotic behavior.

ID	Priority	Incremental finding	Evidence
N01	High	Ambient assumptions can falsify the source-domain component of an inverse certificate.	Native counterexample; source trace; isolated repair and valid-root control.
N02	High	Hidden nonreality produces a trigonometric remainder that is mathematically false.	Native public path; exact growth proof; coefficient-guard controls.
N03	Medium	Optional <code>SeriesData</code> export lacks machine-index preflight after its density fix.	Two one-slot public examples; native diagnostic controls; tested range guard.

These are three findings, not a rebranding of the entire existing backlog. N01 materially strengthens the previously recorded ambient-assumption issue. N02 resolves a prior source-level realness concern with an executed wrong-bound example. N03 identifies a distinct failure that remains after the old dense-allocation defect was addressed. Section 2 makes those relationships explicit.

1.2 Positioning relative to Wolfram Language

The right description is a specialized, inspectable generalized-series and inverse-expansion layer built *with* Wolfram Language. It is not a substitute for the host's complete asymptotic system. Native `Series`, `InverseSeries`, `ComposeSeries`, `Asymptotic`, and `AsymptoticSolve` already cover substantial parts of the domain. The repository adds selected sparse representations, inverse algorithms, explicit remainder transport, retained exact cores, and branch-aware workflows. Its special-function adapter itself consumes native series output.[\[19, 21, 22, 23, 24, 12, 1\]](#)

Two direct comparisons illustrate why claims of uniqueness need care. For $x + x^3$, native reversion and the package return the same finite inverse polynomial. For $\zeta(x)$ at positive infinity, native `Asymptotic` and the package both return the first four Dirichlet terms. The package's explicit

tail representation and additional bound metadata are useful, but the finite expansion is not beyond the native system. No speed advantage was measured in this audit.

2 Snapshot, methodology, and prior-review filtering

2.1 What is pinned and what was executed

All repository claims refer to commit `921387e5ba1239bfda96e63e64e89bf63d9c41e6`, not a moving `main`. The README identifies package version 1.8.0 and Wolfram Language 15.0 or later. It distinguishes the canonical modular kernel from the generated root-level standalone file and documents regeneration through `validation/build_standalone.py`.^[1, 6]

Successful native calls reported

```
15.0.0 for Linux x86 (64-bit) (May 6, 2026)
```

The pinned standalone was loaded successfully, including by the documented download-then-`Get` approach. An imported text probe reported 574,893 characters; repository metadata reported 574,894 bytes. Those are different measures, not inconsistent checksums. A separate file-hash request failed at the service layer, so this report does not invent a package SHA-256. The reproduction runner computes one from the local file it actually loads.

The evaluator was intermittent: some requests failed with transport or upstream-service errors, while subsequent calls succeeded. A transport failure is not an algorithmic timeout, an incorrect result, or a negative test. Only successful responses are treated as native observations. The strongest baseline certificate example was reconfirmed after normal `Get[URLDownload[...]]`, and the native numerical checker was tested against the same invalid branch target in that call.

2.2 Evidence classes and limits

Observed native behavior means an output was returned by the connected Wolfram evaluator for the pinned package or a stated focused modification. **Mathematical deductions** are the proofs in this article, including the trigonometric growth contradiction and the native-grid index calculation. **Source deductions** identify the inspected route responsible for a behavior. **Candidate designs** are proposed changes, not claims about code already present.

The archive contains 23 executed independent Python checks covering exact arithmetic facts, grid specifications, and patch-transformation fixtures. They passed. They are not an implementation of Wolfram evaluation, a substitute for native tests, or evidence that every package module works. The supplied eight-test Wolfram regression file and its runner were lexically checked for balanced delimiters, but were not executed as a complete file. Individual probes corresponding to its central contracts were executed separately. The combined patch was not run against the upstream suite.

This audit did not clone and test every repository artifact locally, prove all modules correct, or read every historical report line by line. Source inspection concentrated on the main kernel, certificates, ordinary and envelope arithmetic, source/domain transport, native and selected specialized forward paths, and supporting inverse/flat/Fourier modules. Appendix A identifies that coverage. These limits matter: three concrete defects do not imply that unrelated numerical formulas are wrong, and passing controls do not prove the absence of other defects.

2.3 How duplication was avoided

The nine-review index and consolidated `CODE_REVIEW_STATUS.md` supplied the baseline inventory. The relevant original arguments were checked in review 7’s detailed findings and review 8’s audit overview. Existing items were used as exclusion criteria and as pointers to questions needing sharper evidence.[2, 3, 4, 5]

This audit	Closest earlier item	What is genuinely added
N01	Consolidated C05; review 8 F01	A false <i>certificate</i> , not only lost assumption metadata; identical saved input tested under two contexts; numerical-check disagreement; an isolated fix with a positive control.
N02	C07 and related C13; review 7 F04	A native algebraic-number encoding bypass, followed by a provably false $\mathcal{O}(1/x)$ bound. The old concern was not an executed public wrong-bound example.
N03	C01; review 7 F01; review 8 F03	The current density cap is acknowledged. The new issue is native integer representability, demonstrated with exactly one retained lattice slot.

The recently repaired pure-remainder fractional-power guard is not reported again as an unfixed defect. Existing concerns about logarithmic-tail precision, refinement demand, object schemas, broad resource accounting, generic CI, integration, complex sectors, and multivariate extensions are not recycled into new findings or a long recommendation list. The proposals below are narrower: each has a specific obligation exposed by one of the three counterexamples.

3 Architecture and comparison with built-in capabilities

3.1 The ordinary mathematical representation

The ordinary engine uses a positive local coordinate $w \rightarrow 0^+$ and sparse blocks

$$w^\beta P(\log w), \quad \beta \in \mathbb{R},$$

with exact admissible weights and polynomial logarithmic coefficients. Finite and infinite endpoints are reduced to coordinates such as $x - x_0$, $x_0 - x$, $1/x$, or $-1/x$. Internally a jet consists of retained rows and a precision pair:

$$J = (T, \rho, d), \quad f(w) = \sum_{(\beta, P) \in T} w^\beta P(\log w) + \mathcal{O}\left(w^\rho(1 + |\log w|)^d\right).$$

An exact jet uses $\rho = \infty$. The public `GeneralizedSeries` wrapper can additionally carry an offset, exact prefactor, source equation, conditions, operation recipe, and specialized scale information. `Normal` returns the finite expression, deliberately discarding the remainder contract.[6, 9, 1]

For ordinary inversion, the forward model is normalized to

$$f(u) = y_0 + au^p \left(1 + \sum_{j=1}^m u^{\delta_j} P_j(\log u) \right), \quad \delta_j > 0.$$

The inversion works in a positive target uniformizer and groups equal total weights. This avoids pretending that an arbitrary exact real support fits a single rational lattice. The package exposes

coefficient-level access and retained inverse computations in addition to finite expressions. Those are meaningful interfaces for research workflows that repeatedly extend or inspect an inverse expansion.[6, 9]

The representation also makes the proof obligations unusually visible. Addition and multiplication must transport both input errors; exponentiation needs a vanishing absolute error; logarithms need an appropriate positive leading approximation and relative error; derivatives require derivative information about the remainder. Merely differentiating or composing a finite expression would not establish those contracts. The inspected implementation already contains substantial machinery for these distinctions.[9, 11, 10]

3.2 Selected extensions, not a universal transseries field

The repository has separate routes for logarithmic and source-coordinate reductions, exact-core perturbations, finite flat sectors, logarithmic Fourier coefficients, and Gamma/Barnes inverse scales. Its forward special-function support includes native-series import and dedicated parameterized or defining-sum paths. The README explicitly describes endpoint-specific admitted cases rather than support for every parameter regime of every named function.[1, 15, 16, 17, 12, 14]

This division is sensible. A finite flat-sector calculation has an exponential degree and an inner algebraic precision; an ordinary power–log cutoff does not specify both. An oscillatory coefficient can be bounded without being nonzero, so it cannot automatically support division. A retained exponential prefactor can be exact while its correction bracket is only asymptotic. A single generic term count cannot erase these distinctions. The current specialized representations preserve important information that a finite expanded expression alone would lose.[16, 17, 12, 18]

The same division creates interfaces worth auditing: a specialized result may later enter ordinary arithmetic or an envelope fallback. N02 concerns precisely such a fallback, although its input is a simple ordinary forward series. N03 concerns the opposite direction: converting a sparse representation into a more restrictive native view. The arithmetic formulas inside either representation can be correct while the bridge between their contracts is not.

3.3 Capability matrix

Task	Native Wolfram Language	Repository’s role and limitation
Local Taylor, Laurent, and fractional-power series	Series and SeriesData supply local series and arithmetic.	Adds sparse exact-real weights, polynomial-log blocks, positive real charts, and explicit custom error descriptors. Not every such object has a native view.[19, 20, 6]
Series reversion	InverseSeries reverts suitable native series; its documented scope is not limited to a single ordinary Taylor case.	Adds specialized real-branch inversion and coefficient/provenance access for its admitted models. Ordinary cases overlap.[21, 6]
Composition	ComposeSeries composes native series and tracks their series order.	SeriesCompose transports custom scale information; separate compatibility and domain obligations remain.[22, 9]

Task	Native Wolfram Language	Repository's role and limitation
General asymptotic approximation	Asymptotic can infer nonstandard scales and handles more than elementary power-series substitution.	Offers selected persistent objects and explicit operations rather than only a requested approximation expression. It does not supersede the full native problem set.[23, 1]
Asymptotic equations	AsymptoticSolve addresses equations and systems with real or complex asymptotic specifications.	Primarily specializes scalar real inverse problems and selected reductions. Do not infer that the native solver has no inverse capability.[24, 6]
Special functions	Native Series/Asymptotic are major coefficient sources.	Imports structured native output, separates carriers, and adds selected direct expansions and bounds. Endpoint and fixed-parameter restrictions are important.[12, 13, 14]
Numerical comparison and certification	Numerical root solving is distinct from a proof of an interval enclosure.	Provides separate numerical evidence and a restricted exact interval-certificate path. N01 shows the latter's source-domain claim needs repair.[8, 7]

This table compares responsibilities, not a benchmark league table. There is no evidence here that the package is uniformly faster, more general, or more reliable than native facilities. Conversely, the presence of a native command with a related name does not remove the value of the package's inspectable sparse coefficient algebra and retained contracts.

3.4 Executed overlap: polynomial inversion

Experiment E11 compared

```
s = AsymptoticInverse[x + x^3, {x, 0}, {y, 6}];
n = InverseSeries[Series[x + x^3, {x, 0, 6}], y];
{Normal[s], Normal[n], Simplify[Normal[s] - Normal[n]]}
```

and returned

$$\{y - y^3 + 3y^5, \quad y - y^3 + 3y^5, \quad 0\}.$$

The package remainder was `PowerLogRemainder[y, 7, 0]`. This is a useful control for the core implementation and a warning against marketing ordinary reversion as absent from the host. It does not test every inverse method or prove equality of internal algorithms.

3.5 Executed overlap: a Dirichlet scale

Experiment E12 compared

```
AsymptoticExpansion[Zeta[x], x -> Infinity, SeriesTermGoal -> 4]
Asymptotic[Zeta[x], x -> Infinity, SeriesTermGoal -> 4]
```

Both finite expressions were

$$1 + 2^{-x} + 3^{-x} + 4^{-x}.$$

The package's stored remainder was

$$\text{PowerLogRemainder}[e^{-x}, \log 5, 0],$$

which denotes an $\mathcal{O}(5^{-x})$ tail. Its dedicated Dirichlet path additionally records a first-omitted-integer bound. For fixed integer $m \geq 1$ and $S > 1$, monotone integral comparison gives

$$m^{-S} \leq \sum_{n=m}^{\infty} n^{-S} \leq m^{-S} + \int_m^{\infty} t^{-S} dt = m^{-S} \left(1 + \frac{m}{S-1}\right). \quad (1)$$

This is the bound implemented in the inspected source. It is useful explicit information, but the calculation above is not a new algorithm contributed by this audit.[\[14\]](#)

The correct comparative statement is therefore precise: in this tested case the finite terms agree, while the package offers a particular persistent remainder/bound interface. A comparison against `Series` alone would miss native `Asymptotic`'s ability to return the same exponential scale.

4 N01: Ambient assumptions can invalidate a positive certificate

Priority: high. Evidence: native public counterexample and focused repair controls.

4.1 The counterexample

Construct an inverse object in a clean assumption context:

```
s = Block[{$Assumptions = True},
  AsymptoticInverse[
    ConditionalExpression[x, x > 100],
    {x, Infinity}, {y, 2}]];
```

This describes the identity function on the retained source domain $x > 100$, approached at positive infinity. The saved object retains `Function -> x`, `Expression -> y`, and `SourceDomain -> x > 100`. The following requested interval is deliberately outside that domain:

```
Block[{$Assumptions = True},
  InverseCertificate[s, 2,
    "Interval" -> {1, 3}, "Center" -> 2,
    "MaxRefinements" -> 0]]
```

The clean-context output is correctly `Failure["OutsideBranch",...]` with the source-domain condition unproved over the entire closed interval.

Now change only the caller's ambient assumptions, not the saved object, target, center, or interval:

```
Block[{$Assumptions = x > 100},
  InverseCertificate[s, 2,
    "Interval" -> {1, 3}, "Center" -> 2,
    "MaxRefinements" -> 0]]
```

The native output includes

```
<|"Certified" -> True,
  "RootEnclosure" -> {2, 2},
  "CertifiedSourceDomain" -> x > 100,
  "SourceDomainVerified" -> True,
  "CertifiedTarget" -> 2,
  "ExistenceEvidence" -> "Residual bracket containment", ...|>
```

This is not merely a misleading numerical name or an overly broad reading of a local theorem. The output explicitly says that the retained source domain was verified. Every point in $[1, 3]$ violates $x > 100$, and in particular the alleged root does. Experiment E02 records the result; E10 reconfirms it alongside the numerical-check control.

4.2 What is correct, and what is false

The unrestricted identity equation has root 2. Its derivative is 1 and its center residual is zero. Thus the rational interval arithmetic has an easy, correct equation-solving problem. The error is in the additional domain proposition attached to that solution.

Proposition 4.1. *Let $D(x)$ be the retained source condition and I the verification interval. A certificate claiming a unique root of the restricted source in I must establish the relevant domain obligation, here $\forall x \in I D(x)$, from the recorded premises. Assuming $D(x)$ during that check does not establish the obligation.*

Proof. For this example take $D(x) \equiv x > 100$ and $I = [1, 3]$. The quantified domain obligation is false, independently of the identity equation. Simplifying $D(x)$ under premise $D(x)$ proves only the tautology $D(x) \Rightarrow D(x)$, not that the interval lies in the domain. The returned root 2 therefore cannot satisfy the claimed restricted-source certificate. $\square$

The package's own scope text promises a root on the selected source side and within retained source-domain conditions. N01 contradicts that promise on a public path; it does not demand a global inverse theorem that the implementation never offered.^[7]

4.3 Root cause: positional assumptions are not isolation

The inspected certificate helper `certPositiveCondition` begins by refining a predicate under a positional reality assumption for the source variable. The critical operation has the shape

```
Refine[condition, Element[x, Reals]]
```

That form does not neutralize `$Assumptions`. Wolfram documents that the default `Assumptions` option is appended to the positional assumptions; explicitly supplying the option prevents that default contribution.^[25]

Experiment E01 directly observed

```
Block[{$Assumptions = True},
  Refine[x > 100, Element[x, Reals]]      (* x > 100 *)
Block[{$Assumptions = x > 100},
  Refine[x > 100, Element[x, Reals]]      (* True *)
```

Once the condition becomes `True`, the later interval test no longer has the original inequality to verify. The exact residual and derivative computations cannot recover the lost obligation.

This is a stronger consequence than the old observation that construction could use ambient assumptions while recording `Assumptions -> True`. Here the object was constructed before the problematic ambient assumption was introduced. Fixing constructor metadata alone would leave this verification-time counterexample intact.^[5, 3]

4.4 A revealing differential control

Under the same ambient assumption, the numerical checker rejects target 2:

```
Block[{$Assumptions = x > 100},
  InverseNumericalCheck[s, 2, WorkingPrecision -> 20]]
(* Failure["OutsideBranch", ...] *)
```

It substitutes the recovered numerical root into the source condition and recognizes the failure. Thus the certificate, which makes the stronger claim, accepts a case that the noncertifying numerical check rejects. This is not proof that the numerical route is generally sounder; it is an excellent targeted regression relation.^[8]

4.5 Focused repair and validation

The candidate change isolates the entire public verification body:

```
InverseCertificate[GeneralizedSeries[a_Association], yv_, opts___] :=
  catch[Block[{$Assumptions = True},
    Module[(* existing locals *),
      (* existing verification body *)]]]
```

The listing is schematic; the accompanying patch rules preserve the actual option pattern and body. Whole-body isolation is preferable to changing one visible `Refine`, because other simplification or branch-reduction helpers called during verification can otherwise consult the same ambient state. It is also preferable to copying `$Assumptions` into certificate metadata: recording an unjustified premise would not turn an empty-domain interval into a valid one.

Experiment E07 tested this focused modification. The target-2 case returned `OutsideBranch` under `$Assumptions = x > 100`. A positive control with target 102, interval `{101, 103}`, and center 102 still returned a positive certificate with enclosure `{102, 102}`. The repair did not simply disable all certification.

This is a narrow repair, not protection against arbitrary redefinition of Wolfram primitives or corruption of saved objects. It also cannot repair an object whose coefficients were already simplified under unrecorded assumptions during construction. Those are separate boundaries. The immediate claim is only that verification must not import fresh ambient assumptions as proofs of interval obligations.

4.6 Acceptance criteria

For a saved valid object and fixed exact target/interval, changing unrelated caller assumptions must not change the truth of its certificate. A source-domain predicate must remain an obligation until proved on the interval; it must not be a premise that discharges itself. Regression variants should include contradictory ambient assumptions, predicates involving the source variable, parameter-only assumptions, and calls from `Assuming`. Positive controls must remain available so that blanket rejection is not mistaken for correctness.

The package should distinguish two policies explicitly. During *construction*, a documented policy

may capture selected caller assumptions and store them with the result. During *verification*, the already recorded premises are the input; any additional source-domain predicate is a proposition to prove. Mixing these policies is the essential error, not the mere presence of a global Wolfram variable.

5 N02: Nonreal admission followed by a false real-only error transport

Priority: high. Evidence: native public path and a mathematical contradiction.

5.1 An exact coefficient without an explicit complex literal

Set

```
c = Root[5 + 5*# + #^5 &, 2];
{Head[c], FreeQ[c, _Real | _Complex], Element[c, Reals]}
(* {Root, True, False} *)
```

The exact algebraic number is nonreal, but its expression tree has no machine-real or `Complex` literal. The tested numerical value was approximately

$$c = -0.8026283840948295 - 1.1850818593889156i.$$

The approximation is illustrative; the exact `Element[c, Reals] -> False` observation is what matters.

There is also a simple algebraic explanation. The polynomial $p(t) = t^5 + 5t + 5$ is irreducible by Eisenstein's criterion at 5. For real t ,

$$p'(t) = 5(t^4 + 1) > 0,$$

and $p(-1) < 0 < p(0)$. It therefore has exactly one real root. Wolfram's documented ordering puts real roots first; index 2 is consequently nonreal.^[26]

The current helper `exactQ` checks absence of `_Real | _Complex`. That is an exactness/encoding check, not a semantic proof of reality. The public validation message speaks of exact real input, but the ordinary constant admission route does not consistently establish that stronger property.^[6]

5.2 The public calculation

The first operation deliberately discards the coefficient into a remainder:

```
s = AsymptoticExpansion[c/x, {x, 0, -2}];
```

The finite support below the exclusive cutoff -2 is empty. The returned object includes

```
"Expression" -> 0
"Remainder" -> PowerLogRemainder[x, -1, 0]
"TargetDomain" -> x > 0
```

Its original source is still c/x . Consider now the ordinary public operation

```
t = Sin[s];
```

Experiment E04 returned a composite object with

```
"Expression" -> 0
"Remainder" -> PowerLogRemainder[x, -1, 0]
"RemainderScaleExpression" -> x^(-1)
"Scale" -> "Composite"
```

and the composite majorant contract asserting that the represented function differs from the finite expression by the recorded asymptotic envelope.

The initial magnitude statement $c/x = \mathcal{O}(1/x)$ would be valid even in a complex-valued interpretation. The false step is the conclusion that $\sin(c/x) = \mathcal{O}(1/x)$. This makes N02 stronger than a complaint that the implementation has violated its advertised real-only API: the returned bound is not rescued by allowing complex values.

5.3 Proof that the remainder is wrong

Proposition 5.1. *For every fixed $c = a + ib$ with $a, b \in \mathbb{R}$ and $b \neq 0$,*

$$\sin(c/x) \notin \mathcal{O}(1/x) \quad (x \rightarrow 0^+).$$

Proof. The elementary identity for the complex sine gives

$$|\sin(a/x + ib/x)|^2 = \sin^2(a/x) + \sinh^2(b/x).$$

Hence

$$x |\sin(c/x)| \geq x \sinh(|b|/x).$$

Writing $t = |b|/x$, the right-hand side is $|b| \sinh(t)/t$, which tends to infinity. Thus no fixed constant can bound $|\sin(c/x)|$ by a multiple of $1/x$ near zero. $\square$

No delicate cancellation or high-order coefficient computation is involved. The counterexample consists of one algebraic coefficient, one monomial, one truncation, and one unary observable. It exposes a missing premise of an otherwise valid real inequality.

5.4 Source path and the lost information

In the ordinary forward engine, an expression independent of the local variable enters **pConst**. The inspected definition verifies polynomial dependence on the logarithm placeholder and performs zero handling; it does not require every admitted coefficient to be real. Consequently the nonreal **Root** reaches the jet.[6]

After truncation, the known finite expression is 0. The ordinary unary route cannot treat the unbounded remainder as a small Taylor increment and uses the allowed envelope fallback. There, **seriesEnvelopeUnary** checks that the *finite approximation* is real. For **Sin** and **Cos**, it then preserves the error using their real-line Lipschitz property. The reality of the unknown represented function, rather than just 0, has not been established.[10, 11]

The logical distinction is

$$\text{Expression}(s) \in \mathbb{R} \not\Rightarrow \text{Realization}(s) \subseteq \mathbb{R}.$$

A pure remainder has no retained coefficient from which its codomain can be recovered. Truncation must not erase a condition needed by later operations. Checking only surviving rows is therefore intrinsically insufficient, even if that check is implemented perfectly.

This sharpens review 7's F04, which identified the missing realness gate and proposed a symbolic logarithm test, and the related concern that unknown remainders cannot inherit properties from finite approximations. The present example is native, uses an exact numeric coefficient, reaches a public arithmetic operation, and has a proved false output bound.[\[4, 3\]](#)

5.5 A focused guard, with its compatibility caveat

The supplied candidate inserts a semantic check in `pConst`:

```
If[! And @@ (
  TrueQ[Refine[Element[#, Reals], ass, Assumptions -> True]] & /@
  CoefficientList[c, ell]),
fail["UnprovedRealCoefficient",
  "Every admitted coefficient must be real under the explicit assumptions.",
  <|"Coefficient" -> c|>]];
```

The explicit option prevents the guard from silently accepting fresh ambient assumptions. Experiment E08 applied the equivalent guard to the fully qualified private definition. The nonreal `Root` source was rejected with `UnprovedRealCoefficient`. Ordinary e^x still produced $1 + x + x^2/2$, and ax with explicit $a \in \mathbb{R}$ assumptions was accepted.

This is a *guard demonstrator*, not a complete semantic-domain implementation. A strict check can reject formerly accepted symbolic coefficients unless their realness is explicitly provable. Specialized algorithms may use complex intermediate expressions before a justified real projection, so placing a universal realness requirement in every low-level algebra would be too crude. The intended invariant must be attached to the particular real coefficient algebra, not imposed indiscriminately on the Fourier or native complex-amplitude machinery. The archive does not claim that the combined guard has passed all those paths.

A production repair should decide where the strict real contract is established, then audit every constructor and adapter that can create an object consumed by the real-only envelope rules. It should not repair this example by inspecting the discarded `FrontierTerm` alone: future errors may originate from an unknown tail rather than one visible coefficient.

5.6 Whole-germ codomain evidence

A useful next contract is a small, explicit classification such as

```
"ValueDomain" -> "Real"
"ValueDomainEvidence" -> <|
  "Scope" -> "WholeRepresentedGerm",
  "Assumptions" -> recordedAssumptions,
  "Source" -> admissionOrTransportRecord|>
```

This is a proposed design, not an existing property. The key phrase is *whole represented germ*. Exact real arithmetic preserves it; certified real unary maps can preserve it; truncation preserves it because truncation changes available approximation data, not the underlying function. Unknown or complex codomain must not be silently upgraded because a finite expression happens to become real.

For a real germ $f = e + r$, the inequality

$$|\sin f - \sin e| \leq |r|$$

is valid without requiring $r \rightarrow 0$. That explains why the existing fallback is valuable and should not simply be deleted. For a general complex germ it is invalid; a different estimate involving

imaginary parts or a controlled complex neighborhood is required. The simplest supported policy is to reject the real-only transport when whole-germ reality is absent. General complex support is a separate existing roadmap topic, not a necessary precondition for fixing N02.

6 N03: Native index range is independent of sparse support size

Priority: medium. **Evidence:** native public examples and tested conversion preflight.

6.1 What the earlier repair already accomplished

The current `makeRationalSeriesData` computes a rational exponent lattice, limits the retained coefficient span to 100,000 slots, and does not pad the coefficient list all the way to the remainder index. That addresses the earlier eager-dense-allocation problem. This audit does not repeat the old allocation example as though the repair were absent.[\[6, 3, 4, 5\]](#)

There is a separate target-format restriction. A native `SeriesData` object has integer start and remainder specifications and a positive integer power denominator. The successful native diagnostic probes reported that these specifications must be machine-sized integers. A tiny sparse support can require a large value in one of those fields even when its coefficient list has length one.

6.2 Two one-slot examples

Let

$$Q = 2^{100} = 1267650600228229401496703205376.$$

First compute

```
a = AsymptoticExpansion[1 + x^Q, {x, 0, 2}];
```

The finite part is 1. The exact first omitted monomial supplies remainder exponent Q , even though the requested cutoff was only 2. The optional native view has the specification

```
SeriesData[x, 0, {1}, 0, Q, 1]
```

with one coefficient. The remainder index is outside the tested native machine-integer range.

Second compute

```
b = AsymptoticExpansion[
  x^(1/Q) + x^(3/Q), {x, 0, 2/Q}];
```

The finite part is $x^{1/Q}$, and the remainder exponent is $3/Q$. The native index triple is $\{1, 3, Q\}$. Again there is one retained slot, but now the denominator is not representable.

Experiment E06 observed both public objects retaining invalid native `SeriesData` forms instead of a structured unavailable-view result. Separate direct native probes E05 emitted `SeriesData::sdatn` for oversized order specifications and `SeriesData::sdatd` for an oversized denominator. A large starting index was also checked directly. No large array was allocated to demonstrate any of these failures.

6.3 The precise conversion obligation

For retained rational exponents S and rational remainder exponent ρ , define

$$d = \text{lcm}\{\text{den}(\beta) : \beta \in S \cup \{\rho\}\}, \quad n_{\min} = d \min S, \quad n_{\max} = d\rho.$$

For nonempty S , the retained dense span is

$$N_{\text{slots}} = d \max S - n_{\min} + 1.$$

The empty-support case has no coefficient slots and a separately chosen starting index. These are the quantities calculated by the inspected helper.[6]

The first witness has

$$(n_{\min}, n_{\max}, d, N_{\text{slots}}) = (0, Q, 1, 1),$$

and the second has

$$(n_{\min}, n_{\max}, d, N_{\text{slots}}) = (1, 3, Q, 1).$$

A condition on N_{slots} cannot imply a bound on $n_{\max}$ or d . Nor does a small requested cutoff imply a small exact first-omitted exponent. The density cap and the integer-range preflight address logically independent properties.

6.4 Impact and severity

The sparse finite expression and its custom remainder remain mathematically useful. This finding does not establish incorrect coefficients in the main expansion. The failure is that an *optional* native view can emit an error or remain an invalid expression while the constructor is otherwise succeeding. Callers using **Check**, expecting a usable **SeriesData** property, or forwarding that property to native operations can be disrupted by information they never needed for the sparse computation.

This is why a medium priority is appropriate rather than labeling it another high-severity allocation defect. The old density repair removed the large temporary array risk in these one-slot cases. What remains is a reproducible representability and API-totality gap at the adapter boundary.

6.5 Focused range guard

Before allocating or constructing the native view, the candidate checks all three indices against the runtime signed machine-integer interval:

```
If[! And @@ (
  IntegerQ[#] &&
  -2^($SystemWordLength - 1) <= # < 2^($SystemWordLength - 1) & /@
  {nmin, nmax, den}) || den <= 0,
  Return[Missing["NativeSeriesDataIndexRange",
    <|"Indices" -> {nmin, nmax, den},
    "SystemWordLength" -> $SystemWordLength|>], Module]];
```

This leaves the sparse object intact and makes the unavailable conversion explicit. The diagnostic separates an index-range refusal from the existing density refusal; the two should not share a misleading failure label.

Experiment E09 applied the source insertion to a downloaded standalone, wrote a temporary patched file, and loaded it with **Get**. On the 64-bit native kernel, both witnesses returned an unavailable-view **Missing** value identifying the out-of-range index. Their finite parts were unchanged. The ordinary e^x control retained a valid native view:

```
SeriesData[x, 0, {1, 1, 1/2}, 0, 3, 1]
```

The guard has not been certified on every platform or at every extreme accepted by the host representation. Boundary regressions should include the smallest and largest machine-sized values, negative starting orders, empty support, ordinary ramification, and safe refusal of large denominators.

6.6 Performance implication beyond this example

A representation adapter needs at least three independent preflights: mathematical representability in the target scale, resource feasibility of the target coefficient layout, and representability of the target’s metadata integers. Counting terms alone addresses none of these completely. That statement is not a call to repeat the earlier broad resource-budget recommendations; it specifies the missing native-view test exposed by N03.

The bit length of an exponent is a different input dimension from the number of nonzero blocks. A useful targeted benchmark varies those dimensions independently: hold support size at one while increasing the remainder index; hold the lattice span at one while increasing the denominator; then hold indices small while widening interior gaps. Record whether conversion succeeds, is refused cleanly, or emits messages. This experiment measures adapter behavior, not inverse-coefficient speed, and can be bounded before any large allocation.

7 A development plan derived from the counterexamples

7.1 First milestone: make verification context-independent

The immediate task is to isolate `InverseCertificate` from fresh ambient assumptions and add the invalid-interval/valid-interval pair. Its completion criterion is specific: E02 must reject the false domain, E07’s valid root must still certify, and E10’s numerical-domain rejection must no longer disagree with a positive certificate on that case.

The next inspection target is every helper called by the certificate that simplifies a condition before substituting an interval or a target. Classify each condition as a recorded parameter premise, a source-domain obligation, an interval-containment fact, or a conclusion. The categories should not be collapsed into one conjunction passed indiscriminately to a simplifier. In particular, the condition being verified must not enter the premise set merely because it appears in `$Assumptions`.

A useful design rule is

$$\text{Verify}(\text{saved data}, \text{request}; A_{\text{ambient}}) = \text{Verify}(\text{saved data}, \text{request}; \text{True})$$

for all caller contexts that do not change the explicit arguments themselves. This is a proposed noninterference property, not a claim that Wolfram evaluation is immune to arbitrary symbol definitions. It is deliberately narrower and testable.

Saved objects can contain legitimate parameter assumptions. The checker should use those as recorded conditional premises, or require parameters to be instantiated when the interval evaluator needs numeric constants. It should not erase legitimate recorded information while blocking external context. Conversely, a condition involving the interval variable is not a numerical parameter fact and generally must be verified throughout the interval. These distinctions are the added engineering requirement beyond the old recommendation to retain assumption metadata.

7.2 Second milestone: establish a real-germ boundary

The immediate N02 guard closes the demonstrated ordinary coefficient route. The durable task is to define where realness is proved for every object accepted by real-only arithmetic. That proof must concern the exact source or the allowed unknown remainder, not only the visible approximation. A constructor that cannot establish its promised real domain should fail or return an explicitly weaker object that real-only operators refuse.

For native special-function import, the implementation already contains deliberate real-source and real-projection checks. Those should supply codomain evidence rather than forcing later consumers to guess from the simplified finite output. For flat or Fourier calculations, complex intermediate coefficients can be compatible with a real final germ when the relevant symmetry or projection is established. The proposal is therefore not “reject every occurrence of i anywhere.” It is “require the real-only transport to consume a justified real-germ contract.”[12, 17]

The minimum propagation rules are uncomplicated. Real constants and the positive real local variable establish realness. Real addition and multiplication preserve it. A real logarithm needs positive input; a noninteger power needs its specified real branch. Truncation preserves previously established whole-germ realness but cannot establish it anew. A bounded real observable can then use the correct real inequality. Any native or specialized operation that cannot justify the flag must return unknown rather than defaulting to real.

Acceptance should include all three shapes: a real finite approximation with a real remainder, a real finite approximation with a nonreal discarded contribution, and a complex intermediate representation whose final source is proved real. Only the second must be excluded from the real Lipschitz rule; the first is useful supported behavior, and the third prevents an unnecessarily destructive patch.

7.3 Third milestone: make the native view independently fallible

The N03 preflight should return a reasoned unavailable-view value without changing `Normal[s]`, the custom remainder, or the sparse coefficients. The practical invariant is

$$\text{successful sparse construction} \not\Rightarrow \text{successful native conversion},$$

while failure of optional conversion must not silently become failure of the sparse mathematics.

The view should distinguish irrational support, excessive retained lattice span, out-of-range indices, and other semantic limitations rather than exposing an unevaluated invalid `SeriesData`. The first two categories already have explicit handling in the inspected helper; the new change supplies the third. A completely lazy export interface may remain desirable, but that was already discussed in earlier reviews. N03 does not require waiting for that larger redesign.[6, 4, 5]

For API consumers, an unavailable view is not a numerical or mathematical failure. Document that native consumers must check for a usable `SeriesData` value; custom-series consumers should continue using the original object. The failed conversion’s diagnostic should state which metadata field cannot be represented and leave the original exact value available for inspection.

7.4 A compact regression matrix

The three defects suggest paired tests that ordinary example-by-example output checks miss:

Boundary	Paired experiment	Required invariant
Verification context	Same saved inverse and interval; clean versus source-dependent ambient assumptions	The domain certificate must not change from false to true.
Coefficient encoding	Explicit nonreal constant versus nonreal algebraic Root	Semantic nonreality cannot be accepted merely because one encoding lacks Complex .
Truncation	Same realness evidence before and after all finite rows are dropped	The whole-germ codomain is preserved or remains unknown; it is not inferred from zero.
Native view	Same one-slot layout; small versus huge remainder index or denominator	Sparse expression survives; impossible view is refused with a structured reason.
Positive controls	Valid source interval, ordinary Taylor series, explicit real parameter	Repairs preserve admitted behavior rather than passing by indiscriminate rejection.

The archive’s eight desired-contract tests cover the minimal examples and positive controls, not every row of this extended matrix. The matrix is a concrete follow-on testing specification, not a statement of tests already run. It can be applied to both modular and standalone loading so that a correct kernel change is not lost during distribution regeneration.

7.5 Performance work should preserve these proof obligations

The proposed changes add different costs. Certificate isolation is a context boundary, not another asymptotic coefficient algorithm. Native index preflight is small integer arithmetic done before allocation. Semantic coefficient admission can be expensive when a symbolic realness question is difficult. That last cost needs a documented policy: accept a proved result, reject a disproved result, and distinguish unresolved or budget-exhausted proof attempts from mathematical nonreality.

A cache for realness or domain results must include the complete relevant assumptions and expression, and must not reuse a result proved under a stronger context as though it were unconditional. This is not a general recommendation to add more caches; it is a restriction any optimization of the new semantic checks must satisfy. Caching a **True** answer without its premises would recreate N01 in a different form.

For the same reason, performance measurements should report which proof obligations were discharged, not only elapsed time and the number of retained terms. A faster result produced by skipping a domain check is not an equivalent benchmark. A refusal caused by an undecided proof is also different from an algorithm running out of coefficient budget. No performance numbers for these repairs are claimed here; the native checks established behavior, not comparative speed.

8 What this audit does not add to the backlog

8.1 Known issues that are not new findings here

The consolidated status already covers multiple precision, refinement, resource, API, distribution, and future-feature topics. Repeating that list would increase the apparent breadth of this article

without adding evidence. In particular, this audit does not claim a new general logarithmic-tail defect, a new generic `MaxTerms` bypass, a new whole-suite infrastructure gap, or a new need for complex/multivariate transseries. Those issues have existing homes.[3]

The source-level concern about the old `fwdSeries` fallback’s unseen logarithmic tail was inspected, but no new public wrong-bound example was established here. It remains a previously recorded issue, not a fourth finding. Likewise, source-domain transport through arbitrary series composition deserves careful treatment, but the current article’s confirmed domain counterexample is specifically the certificate path. These distinctions prevent plausible risks from being presented as executed bugs.

8.2 Corrections to overly broad assurances

The earlier certificate discussion correctly explains residual and derivative enclosure mathematics and the intended interval-domain check. N01 shows that the existence of those checks in source is not sufficient to assert the implemented domain certificate is always enforced. The narrower statement is that the interval arithmetic supplies the stated conclusion *provided the verification context cannot discharge a domain obligation by assuming it*. The counterexample supplies the missing qualification.[4, 7]

Similarly, accepting a nonreal coefficient is not merely harmless extra generality outside a real-only specification. Once the object can reach a real-only bound, the omission has a false mathematical consequence. N02 supplies that consequence without depending on an exotic special-function expansion. Finally, the recent density guard should be credited rather than dismissed: N03’s counterexample works precisely because coefficient storage is small and the remaining restriction is a different one.

8.3 No unsupported generalization from the findings

Nothing in these experiments establishes that ordinary inverse coefficients, the Gamma/Barnes expansions, or the explicit Dirichlet tail formula are generally wrong. The successful polynomial and zeta comparisons are controls, not broad validation. Likewise, the focus on certificate soundness does not convert an ordinary package review into a claim of security isolation against malicious kernel definitions. The required contract is mathematical reproducibility from explicit stored data within a normally functioning Wolfram environment.

A reasonable release decision is to fix these demonstrated paths, run the relevant and cross-cutting native tests, and retain a precise scope statement. It is not necessary to solve every previously listed research extension before providing a reliable specialized package. The distinction between an admitted answer with justified metadata and an honest unsupported result remains the right boundary.

9 Artifact guide and reproducibility

9.1 Files in the archive

The article source and PDF are in `article/`. The machine-readable record `evidence/native_observations.json` contains selected successful outputs labeled E01–E12. It is explicitly a curated transcription, not a fabricated complete session log. The snapshot and novelty ledger are beside it. The Python test output is `evidence/python-checks.txt`.

The code directory contains a bounded native reproducer, eight desired-contract tests, a fail-closed test runner, and a guarded source-change emitter. The emitter reads the two affected canonical

kernel files and verifies unique exact anchors before writing patched *copies* to a new destination. When the input is a Git checkout, it also requires the expected revision. It does not alter the checkout, upload changes, or overwrite the standalone distribution.

article/	LaTeX source, references, rendered PDF
code/	Native probes, regressions, patch emitter, Python checks
evidence/	Native excerpts, snapshot, novelty ledger, check output
patches/	Candidate-change scope and validation limitations

9.2 Run the native probes on an explicit local package

Use a clean kernel and an independently obtained copy of the pinned package. The supplied runner never downloads or silently replaces that package:

```
wolframscript -file code/reproduce.wl \
  /absolute/path/to/AsymptoticInverse.wl observations.json

wolframscript -file code/run_regressions.wl \
  /absolute/path/to/AsymptoticInverse.wl test-results.json
```

The first command records behavior, messages, kernel version, elapsed time, and a hash of the local file. It bounds each observation in time and memory. The second evaluates the desired contracts and returns a failing process status unless the expected test report is obtained with all eight tests passing. Some desired contracts are expected to fail on the pinned baseline; that is their purpose.

These delivered scripts were not run end to end through a local Wolfram executable in this environment. The successful observations were obtained as separate remote native calls. The distinction is preserved so that a user does not mistake the supplied files for a completed full regression report.

9.3 Generate and review the candidate source changes

```
python code/apply_candidate_patch.py \
  /absolute/path/to/pinned-checkout \
  --output /absolute/path/to/new-candidate-directory
```

The output consists of modified copies, a unified diff, and a hash manifest. An unexpected source revision, missing anchor, duplicate anchor, or destination inside the checkout is refused. The rule file uses explicit source anchors rather than approximate regular-expression edits across unrelated definitions.

After reviewing and applying the canonical-source changes, regenerate the standalone with the repository's builder. Test both loading forms. Changing only the generated root file would leave the modular implementation inconsistent; changing only the modules without regeneration would leave the advertised standalone unchanged. That distribution relationship is already documented upstream.^[1]

The N01 and N03 focused source modifications were tested natively; N02's equivalent private guard was tested through a fully qualified redefinition. They were not tested together against the full package. In particular, the semantic realness guard needs coverage of parameter assumptions, special-function projections, and internal complex working representations before release qualification.

9.4 Independent checks and article build

```
(cd code && python -m unittest test_independent -v)
make pdf
```

The 23 Python checks use standard-library exact rational and integer operations plus source-transformation fixtures. They verify the one-slot grid calculations, machine-range predicates, the identity-domain counterexample, elementary polynomial facts, and patch-anchor behavior. The divergence proof in Proposition 5.1, not numerical sampling, establishes the false remainder. None of these tests executes Wolfram package code.

The article is built with pdfLaTeX and `latexmk`. All required LaTeX sources are included; no proprietary fonts or external figures are needed. The PDF was compiled and visually inspected after rendering. Build intermediates are omitted from the final archive.

10 Conclusion

The repository’s strongest contribution is not just another list of asymptotic terms. It is an attempt to make specialized inverse expansions usable as objects with explicit mathematical obligations. This incremental audit identifies three places where those obligations are lost across boundaries: caller context becomes verification evidence, the finite approximation’s codomain substitutes for the whole function’s codomain, and sparse mathematical data are assumed to fit native integer specifications.

The counterexamples are deliberately small and decisive. The domain $x > 100$ does not contain root 2. A nonreal coefficient divided by x produces exponentially growing complex sine, not an $\mathcal{O}(1/x)$ remainder. A one-entry coefficient array does not make 2^{100} a machine-sized integer. Each finding has a focused repair, a positive control, and a clear statement of what remains to be validated.

The resulting development recommendation is correspondingly narrow: isolate verification premises, preserve whole-germ semantic evidence, and make optional native conversion explicitly fallible. These changes strengthen the package’s distinctive value without pretending to replace the broad native Wolfram asymptotic system or republishing the first nine reviews.

A Source coverage and navigation

All paths below are relative to the pinned repository. Function names are the primary locators; later revisions may move line numbers. This is an inspection map, not a claim of complete formal verification.

Kernel file or source group	Inspected responsibility
<code>AsymptoticInverse.wl</code>	Exactness/realness helpers, <code>pConst</code> , sparse precision algebra, ordinary forward dispatch and object construction, <code>makeRationalSeriesData</code> , inverse/public interfaces.
<code>InverseCertificates.wl</code>	<code>certPositiveCondition</code> , domain checks, residual/derivative certificate construction, public verification body and scope.
<code>NumericalInverseChecks.wl</code>	Recovery of the numerical source root and retained source-domain rejection.

Kernel file or source group	Inspected responsibility
<code>SeriesOperations.wl</code>	Ordinary observables, composition, differentiation contracts, truncation and refinement recipes.
<code>SeriesArithmetic.wl</code>	Public unary/binary dispatch and fallback into envelope operations.
<code>SeriesEnvelopeArithmetic.wl</code>	Envelope data, realness checks, unary Lipschitz transport, optional composite behavior.
<code>InverseFunctionExpressions.wl</code>	Public forward routing, conditions and inverse-function composition with jets.
<code>InverseFunctionFamilies.wl</code> , <code>SourceCoordinates.wl</code>	Selected branch families and source-coordinate reconstruction.
<code>NativeSpecialFunctions.wl</code> , <code>ParameterizedSpecialFunctions.wl</code> , <code>SpecialFunctionIdentities.wl</code>	Structured native import, amplitude/phase handling, parameterized local expansions and identity normalization.
<code>DirichletSpecialFunctions.wl</code>	Defining-sum exponential scales and explicit first-omitted-integer bound.
<code>GammaForward.wl</code> , <code>ExactTermination.wl</code>	Logarithmic normalization and original-source exactness checks.
<code>FlatSectorOperations.wl</code> , <code>FourierCoefficients.wl</code>	Distinct sector/inner orders, derivative contracts, finite frequency coefficient algebra.
Review index, consolidated status, review 7 findings, review 8 overview	Duplicate filtering and the precise incremental claims.

B Native evidence index

ID	Experiment	Status
E01	Refine with positional assumptions under two ambient contexts	Observed native
E02	Invalid source-domain interval rejected cleanly, falsely certified under ambient source condition	Observed native
E03	Nonreal Root passes the syntactic no- Complex test	Observed native
E04	Truncate c/x , then apply Sin ; false $\mathcal{O}(1/x)$ output	Observed native
E05	Oversized native order/start/denominator diagnostics	Observed native
E06	Two public one-slot native-view failures	Observed native
E07	Certificate context isolation: invalid rejection and valid-root acceptance	Focused fix observed
E08	Strict real-coefficient guard: nonreal rejection and real controls	Equivalent guard observed
E09	Native-index preflight: structured refusal and ordinary-view control	Focused fix observed
E10	Numerical checker rejects the case the certificate accepts	Observed native
E11	Polynomial inverse finite parts agree with InverseSeries	Observed native
E12	Four zeta terms agree with native Asymptotic	Observed native

The evidence record does not supply elapsed-time claims for experiments whose successful response did not report them. It does not transform service failures into failed mathematical tests. The delivered scripts provide a way to collect fresh, complete local records under a chosen kernel version.

References

- [1] Vladimir Reshetnikov, *Asymptotic repository: Repository README*. Pinned revision 921387e. [README.md](#).
- [2] Vladimir Reshetnikov, *Asymptotic repository: Index of the nine review packages*. Pinned revision 921387e. [code-review/README.md](#).
- [3] Vladimir Reshetnikov, *Asymptotic repository: Consolidated code-review status*. Pinned revision 921387e. [docs/development/CODE_REVIEW_STATUS.md](#).
- [4] Vladimir Reshetnikov, *Asymptotic repository: Review 7: detailed findings and related correctness boundaries*. Pinned revision 921387e. [code-review/code-review-7/article/03-findings.tex](#).
- [5] Vladimir Reshetnikov, *Asymptotic repository: Review 8: repository audit and native observations*. Pinned revision 921387e. [code-review/code-review-8/article/asymptotic_repository_audit.tex](#).
- [6] Vladimir Reshetnikov, *Asymptotic repository: Canonical main kernel*. Pinned revision 921387e. [AsymptoticInverse/Kernel/AsymptoticInverse.wl](#).
- [7] Vladimir Reshetnikov, *Asymptotic repository: Exact inverse certificates*. Pinned revision 921387e. [AsymptoticInverse/Kernel/InverseCertificates.wl](#).
- [8] Vladimir Reshetnikov, *Asymptotic repository: Numerical inverse checks*. Pinned revision 921387e. [AsymptoticInverse/Kernel/NumericalInverseChecks.wl](#).
- [9] Vladimir Reshetnikov, *Asymptotic repository: Ordinary series operations and refinement*. Pinned revision 921387e. [AsymptoticInverse/Kernel/SeriesOperations.wl](#).
- [10] Vladimir Reshetnikov, *Asymptotic repository: Public arithmetic dispatch*. Pinned revision 921387e. [AsymptoticInverse/Kernel/SeriesArithmetic.wl](#).
- [11] Vladimir Reshetnikov, *Asymptotic repository: Composite asymptotic envelopes*. Pinned revision 921387e. [AsymptoticInverse/Kernel/SeriesEnvelopeArithmetic.wl](#).
- [12] Vladimir Reshetnikov, *Asymptotic repository: Structured native special-function import*. Pinned revision 921387e. [AsymptoticInverse/Kernel/NativeSpecialFunctions.wl](#).
- [13] Vladimir Reshetnikov, *Asymptotic repository: Parameterized special-function local expansions*. Pinned revision 921387e. [AsymptoticInverse/Kernel/ParameterizedSpecialFunctions.wl](#).
- [14] Vladimir Reshetnikov, *Asymptotic repository: Dirichlet-series forward expansions*. Pinned revision 921387e. [AsymptoticInverse/Kernel/DirichletSpecialFunctions.wl](#).
- [15] Vladimir Reshetnikov, *Asymptotic repository: Source-coordinate inverse reconstruction*. Pinned revision 921387e. [AsymptoticInverse/Kernel/SourceCoordinates.wl](#).
- [16] Vladimir Reshetnikov, *Asymptotic repository: Flat-sector arithmetic*. Pinned revision 921387e. [AsymptoticInverse/Kernel/FlatSectorOperations.wl](#).
- [17] Vladimir Reshetnikov, *Asymptotic repository: Fourier coefficient algebra*. Pinned revision 921387e. [AsymptoticInverse/Kernel/FourierCoefficients.wl](#).
- [18] Vladimir Reshetnikov, *Asymptotic repository: Gamma-product logarithmic normalization*. Pinned revision 921387e. [AsymptoticInverse/Kernel/GammaForward.wl](#).

- [19] Wolfram Research, *Series*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [20] Wolfram Research, *SeriesData*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [21] Wolfram Research, *InverseSeries*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [22] Wolfram Research, *ComposeSeries*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [23] Wolfram Research, *Asymptotic*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [24] Wolfram Research, *AsymptoticSolve*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [25] Wolfram Research, *Refine*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/Refine.html>.
- [26] Wolfram Research, *Root*, Wolfram Language documentation. Consulted 9 September 2026. <https://reference.wolfram.com/language/ref/Root.html>.